

Отчёт по лабораторным работам 5–6

Среда: OWASP Juice Shop (localhost:3000) | Дата: 11.08.2026

ЛР_5 — Пакет инъекций: SQL-инъекция + DOM-XSS (A05)

Часть 1 — SQL-инъекция при входе (обход аутентификации)

Что сделано:

На странице Login в поле Email введена SQL-инъекция: ' OR 1=1--. Запрос модифицировал WHERE-условие так, что оно всегда истинно. Сервер вернул токен первого пользователя БД — admin@juice-sh.op с ролью admin. Пароль не потребовался.

Доказательства:

- Нормальный неудачный вход: «Invalid email or password.»
- Payload: ' OR 1=1-- (email), anything (password)
- Результат: JWT-токен для admin@juice-sh.op (id=1, role=admin)
- Также работает: admin@juice-sh.op'--
-  Login Admin

Ключевая причина:

Конкатенация пользовательского ввода в SQL-запрос без параметризации. Сервер строит запрос вида: SELECT * FROM Users WHERE email = 'input' AND password = 'hash'. Злоумышленник внедряет ' OR 1=1--, что превращает запрос в: SELECT * FROM Users WHERE email = " OR 1=1--" AND password = 'hash'. Комментарий (--) отсекает проверку пароля.

Потенциальные последствия:

- Полный обход аутентификации — вход под любым пользователем без пароля
- Несанкционированный доступ к административным функциям
- Кража/изменение/удаление данных через UNION-инъекции

Исправление / профилактика:

- Использовать параметризованные запросы (prepared statements) вместо конкатенации строк
- Использовать ORM с безопасными шаблонами (Sequelize parameterized queries)
- Принцип минимальных привилегий для учётной записи БД
- Валидация и санитизация пользовательского ввода

Часть 2 — DOM-XSS через поиск

Что сделано:

В поле Search введён XSS-пейлоад: iframe src="javascript:alert('XSS')". Angular-приложение вставляет результат поиска в DOM без надлежащего кодирования, что позволяет выполнить произвольный JavaScript в браузере жертвы.

Доказательства:

- Поисковый запрос с HTML/JS-тегами возвращается в DOM без экранирования
- REST API /rest/products/search?q=... возвращает результаты, которые рендерятся клиентом без санитизации

-  Client-side XSS Protection (persistedXssUserChallenge) — защита обойдена

Ключевая причина:

DOM-based XSS: клиентский JavaScript (Angular) вставляет пользовательский ввод в DOM через innerHTML или аналогичный небезопасный метод без предварительного кодирования. Серверная санитизация (sanitize-html) не полностью защищает от DOM-XSS.

Потенциальные последствия:

- Кража сессионных cookie и токенов (document.cookie)
- Подмена содержимого страницы (фишинг, дефейс)
- Перенаправление на вредоносные сайты
- Keylogging и кража вводимых данных

Исправление / профилактика:

- Кодировать вывод: использовать textContent вместо innerHTML, DomSanitizer в Angular
- Валидация входных данных на сервере и клиенте
- Content Security Policy (CSP) для ограничения источников скриптов
- Избегать опасных DOM API: innerHTML, document.write, eval

ЛР_6 — Время расплаты: отрицательное количество (A06)

Что сделано:

В корзину добавлен самый дорогой товар — Melon Bike (2999 руб.). Через прямое обращение к API (PUT /api/BasketItems/10) количество изменено на -10. Сервер принял отрицательное значение без проверки. Итоговая сумма корзины: -29988.01 руб. Заказ успешно оформлен с отрицательной суммой — система зачислила деньги на кошелек вместо списания.

Доказательства:

- Товар: Melon Bike (Comeback-Product 2018 Edition), цена 2999 руб.
- Изменённый запрос: PUT /api/BasketItems/10 {"quantity":-10}
- Итог корзины: -29988.01 руб. (1x Apple Juice 1.99 + -10x Melon Bike 2999)
- Заказ оформлен: POST /rest/basket/6/checkout — подтверждение получено
-  Payback Time

Ключевая причина:

Отсутствие серверной валидации количества товара. Клиентский интерфейс не позволяет ввести отрицательное значение, но серверный API принимает любое целое число без проверки границ (quantity >= 1). Бизнес-логика расчёта суммы корзины не защищена от отрицательных значений.

Потенциальные последствия:

- Финансовые потери: зачисление денег на счёт злоумышленника вместо списания
- Манипуляция складскими остатками через отрицательные количества
- Нарушение целостности данных заказов и платежей

Исправление / профилактика:

- Серверная валидация: `quantity >= 1`, только целые числа
- Ограничение на уровне БД: CHECK (`quantity >= 1`)
- Проверка итоговой суммы заказа: `total >= 0` перед списанием
- Использование `decimal` для денежных значений (избегать `float`)
- Централизация логики расчёта цен — не дублировать на клиенте